

Lecture 12: Software Technologies for Stereoscopic Displays

“A VR headset has great hardware. But without software, can it show anything?”

Explain:

Hardware gives capability, but **software makes VR actually work**.
Rendering engines, SDKs, and drivers form the brain of the system.

Rendering Engines (Core Concept)

What is a Rendering Engine?

Simple definition:

A rendering engine is software that generates 2D images from 3D scenes in real time.

Explain role in stereoscopic displays:

- Creates **two views** of the same scene
 - One for left eye
 - One for right eye
 - Handles:
 - Lighting
 - Shading
 - Textures
 - Shadows
 - Post-processing
-

Popular Rendering Engines (give real examples)

Unity Engine

- Most popular for AR/VR development
- Easy to learn
- Built-in XR support

Unreal Engine

- High realism
- Used for cinematic VR
- Advanced lighting (Lumen, ray tracing)

Open-source engines

- Godot
- Blender real-time viewport
- WebXR engines (Three.js, Babylon.js)

Explain:

Engines are responsible for maintaining FPS, managing scenes, and creating immersion.

Why Engines Matter for Stereoscopic Displays

Explain key responsibilities:

Engine Feature	Importance in VR
Dual camera rendering	One per eye
Low latency pipeline	Prevents motion sickness
Frame synchronization	Avoids visual discomfort
Physics + interaction	Makes VR feel real
Optimization tools	Keeps FPS high

Give simple example:

In Unity VR, you don't manually create two cameras — the engine handles stereo rendering automatically.

SDKs and Drivers for VR Displays

What is an SDK?

SDK (Software Development Kit) = a collection of tools, libraries, and APIs that allow developers to build applications for specific devices.

Common VR SDKs:

- **OpenXR** (industry standard)
- **Oculus SDK / Meta XR SDK**
- **SteamVR SDK**
- **Windows Mixed Reality SDK**
- **ARCore (Google), ARKit (Apple)** for AR

Explain simply:

SDKs allow your app to talk to the headset hardware.

Role of Drivers

Drivers are low-level software that allow the operating system and headset hardware to communicate.

Drivers manage:

- Display output
- Sensor input
- Tracking data
- Controller input

Analogy:

Engine = Car

SDK = Steering + pedals

Driver = Engine control unit that connects parts

How Everything Works Together (Big Picture)

Explain complete software pipeline:

1. Developer builds VR app in Unity
2. Unity uses OpenXR SDK
3. SDK communicates with headset drivers
4. Drivers talk to hardware sensors & display
5. Headset shows stereoscopic output

You can draw this simple flow on board:

App → Engine → SDK → Driver → Hardware

This makes students clearly understand architecture.

Recap + Quick Questions

Recap:

- Rendering engines create stereoscopic visuals
- Unity and Unreal are most used engines
- SDKs connect apps to headset features
- Drivers connect software to hardware
- All three are essential for working VR systems

Quick oral questions:

- Why does VR need two cameras in the engine?
- What is the purpose of an SDK?
- What happens if drivers are missing?